Lecture 8: Assembly III — Buffer Overflows & Undefined Behavior

Session 10 · Wed Sep 30, 2026

Contents

That Canary Was Defending Something . . . 2
This Lecture in Three Parts . . . 2
Where We Are . . . 3
The Return Address Is Above Your Array . . . 3
Two Facts That Make the Layout Dangerous . . . 3
Walk Through: 24 Bytes Into an 8-Byte Buffer . . . 4
Walk Through: What `ret` Does With the Garbage . . . 4
`strcpy` Asks the Input How Much to Write . . . 4
The Sender Chooses How Far the Write Goes . . . 5
Run It: AddressSanitizer Finds It . . . 5
Reading the ASAN Report . . . 5
Three Outcomes, One Bug . . . 5
Why Tests Miss It . . . 6
What the Attacker Actually Needs . . . 6
Defense 1: The Stack Canary . . . 6
How the Canary Check Works . . . 7
What the Canary Does Not Catch . . . 7
Defense 2: ASLR . . . 7
ASLR: Same Binary, Two Layouts . . . 7
Run It: Nothing Is Where It Was . . . 7
Defense 3: NX, and W^X . . . 8
Defenses Are Layers, Not a Fix . . . 8
Return-Oriented Programming . . . 8
The Only Real Fix Is the Bound . . . 8
Two Ways the Bounded Copy Goes Wrong . . . 9
The "Safe" Function That Is Not . . . 9
Run It: `strncpy` Does Not Terminate . . . 9
`snprintf` Gives Up a Character to Terminate . . . 10
Undefined Behavior . . . 10
Signed Overflow Is Undefined, Unsigned Is Not . . . 10
Defined, Implementation-Defined, Undefined . . . 10
Why Undefined Is Different in Kind . . . 10
The Compiler Does Not Warn — It Assumes . . . 11
Two Mental Models, and Only One Is Compiled . . . 11
Run It: The Check That Disappeared . . . 11
What Each Build Did . . . 11
Why the Other Two Checks Survived . . . 12
The Linux Kernel, 2009 . . . 12
The Check That Was Not in the Binary . . . 12
Strict Aliasing . . . 12
The Two Reads the Standard Allows . . . 13

Run It: Two Defined Ways to Read the Bits . . . 13
Tools That Find These . . . 13
What Each Tool Checks . . . 13
AI Systems Connection . . . 14
The Same Bug Under the Python . . . 14
The Unit in One Slide . . . 14
The Compiler Is Neither Adversary Nor Safety Net . . . 14
Practice . . . 14
Practice II . . . 15
Practice III . . . 15
Practice — Answers 1–2 . . . 15
Practice — Answers 3–4 . . . 15
Practice — Answers 5–6 . . . 15
Practice — Answers 7–8 . . . 16
Reading . . . 16

That Canary Was Defending Something

Lecture 7 ended with a compiled function that contained instructions we did not explain. Here they are again, with the parts we skipped:

```
movq ___stack_chk_guard@GOTPCREL(%rip), %rax   # get the address of a secret value
movq (%rax), %rax                              # load the secret value itself
movq %rax, 32(%rsp)        # store a copy in the frame, below the return address
...                        # the body of the function runs here
cmpq 32(%rsp), %rcx        # is the stored copy still equal to the secret?
jne  __stack_chk_fail      # if not, abort the program
```

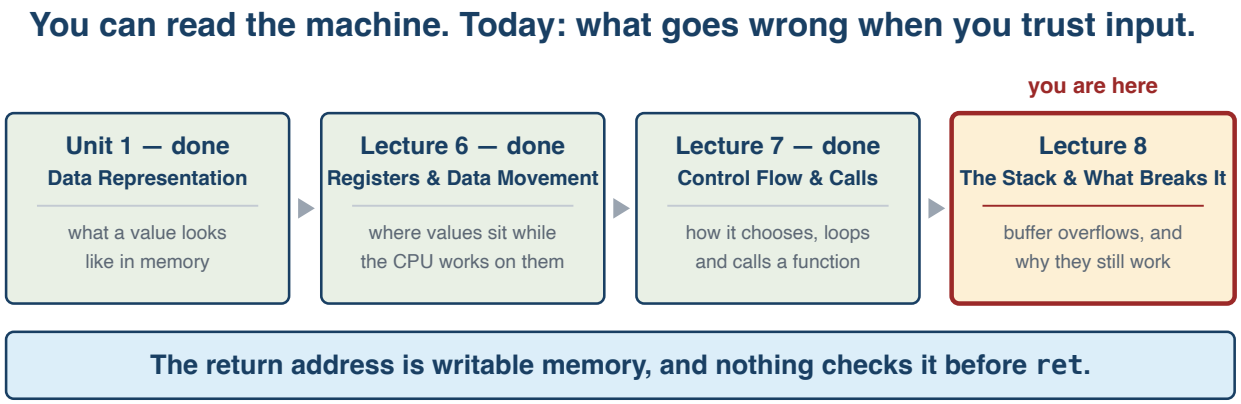
The C source did not ask for any of this. The compiler added it on its own, because the function declared a local **array**. The stored secret is called a **stack canary**, after the canaries that miners carried underground to warn of gas. The compiler is guarding against one specific accident: a write that starts inside the array and keeps going past its end.

This Lecture in Three Parts

This lecture explains that accident. It has three parts.

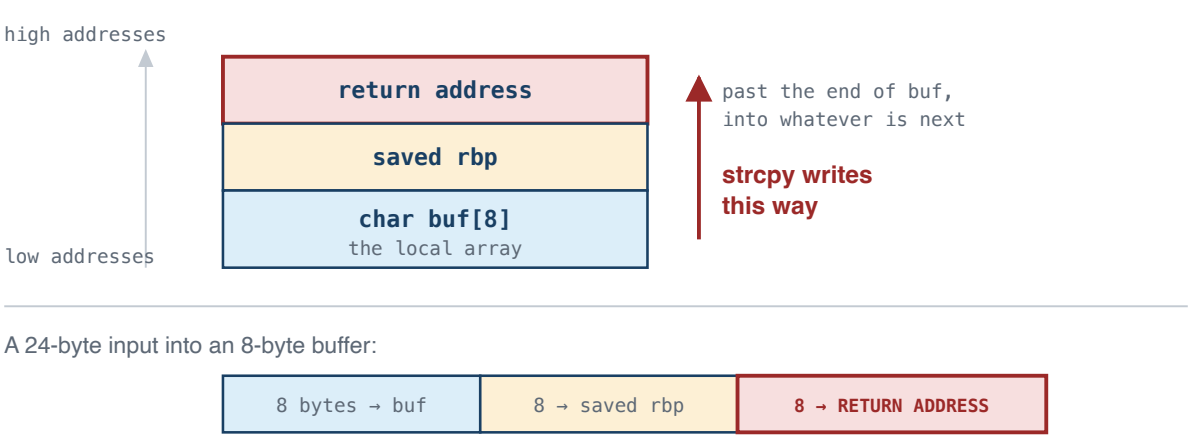
1. **The bug.** What a buffer overflow is, why C makes it easy to write, and what happens to the program when it occurs.
2. **The defenses.** The canary above, address randomisation, and non-executable memory. Each one raises the cost of an attack. None of them removes the bug.
3. **Undefined behavior.** A larger idea that the overflow is one example of. The C standard leaves some operations undefined, and the compiler is allowed to assume they never happen. That assumption can delete code you wrote.

Where We Are



Lectures 6 and 7 taught you to read what the machine does. This lecture is about what it does when the bytes it reads were written by someone else. The machine has no notion of a trusted byte. If a byte sits in the return address slot, `ret` jumps to it. Everything below follows from that.

The Return Address Is Above Your Array



This is the stack frame from Lecture 7, drawn with one local array. Read it from the bottom up, because that is the direction memory addresses increase.

- **`buf`** is a local `char[8]`. It occupies the eight bytes at the lowest addresses in the frame.
- **The saved `%rbp`** sits just above it. Lecture 7's prologue pushed it there so the caller's frame pointer can be restored on return.
- **The return address** sits above that. `call` pushed it, and `ret` will pop it into `%rip` to continue the caller.

Two Facts That Make the Layout Dangerous

Two facts combine to make this layout dangerous.

1. **The stack grows downward, but arrays grow upward.** A new frame is placed at a lower address than the previous one, so `buf` is below the return address. But `buf[0]` is at the lowest address of the array and `buf[7]` is at the highest, so writing an array from start to end moves *toward* the return address.

2. **Nothing separates the regions.** The array, the saved register, and the return address are adjacent bytes in the same memory. The CPU does not know where one ends and the next begins. Only the compiler knew, and the compiler is not present when the program runs.

So a write that begins at `buf[0]` and does not stop at `buf[7]` continues into the saved `%rbp`, and then into the return address. The next slide shows exactly where each byte lands.

Walk Through: 24 Bytes Into an 8-Byte Buffer

Suppose the program copies the 24-character string `AAAAAAAAAAAAAAAAAAAAAAAA` into `buf`. A C string ends with a zero byte, so the copy writes 25 bytes in total. Here is where each byte goes. Offsets are counted from `buf[0]`.

Offset from `buf[0]`	Bytes written	What lived there	What is there now
0 to 7	`A` × 8	`buf` itself	the first 8 characters, as intended
8 to 15	`A` × 8	the saved `%rbp`	`0x4141414141414141`
16 to 23	`A` × 8	the return address	`0x4141414141414141`
24	`0x00`	whatever the caller left above	one zero byte

`0x41` is the ASCII code for `A`. The saved `%rbp` and the return address are now both `0x4141414141414141`, which is not a valid address in any real process.

Walk Through: What `ret` Does With the Garbage

Follow what happens next, one instruction at a time:

1. The function finishes its work and reaches the epilogue.
2. `popq %rbp` restores `%rbp` from the frame. It now holds `0x4141414141414141`. Nothing crashes yet, because `%rbp` is only used as a base for addressing and no address has been computed from it.
3. `ret` pops the return address into `%rip`. `%rip` is now `0x4141414141414141`.
4. The CPU tries to fetch the next instruction from that address. There is no memory mapped there, so the hardware raises a fault and the operating system kills the process. On macOS and Linux the message is a *segmentation fault*.

The important part is step 3. `ret` did exactly what Lecture 7 said it does. It did not check whether the eight bytes it popped were the ones `call` pushed. Nothing in the machine can check that, because a return address is ordinary memory, and ordinary memory holds whatever was last written to it.

Now replace the `A`s at offsets 16 to 23 with a real address of the attacker's choosing. Step 3 then jumps there instead of crashing. That is the whole attack.

`strcpy` Asks the Input How Much to Write

Here is the function whose frame we just traced:

```
void vulnerable(const char *input) {
    char buf[8];
    strcpy(buf, input);     // stops at the input's NUL, not at buf's end
}
```

`strcpy(dst, src)` copies bytes from `src` to `dst`, one at a time, until it copies a zero byte. That is its entire specification. Two things are missing from it.

It does not know the size of the destination. The first argument is a `char *`. A pointer is an address and nothing more, as Lecture 3 showed. The declaration `char buf[8]` recorded the size 8 in the compiler's symbol table, but that number was not stored in memory and is not passed along with the pointer. Inside `strcpy` there is no way to recover it.

It does not take a length argument. So the only thing that ends the copy is the zero byte in the input. Whoever produced the input decided where that zero byte is, and therefore decided how many bytes `strcpy` writes.

The Sender Chooses How Far the Write Goes

Put those together: the number of bytes written into `buf` is chosen by the sender of the input. If the input came from a command-line argument, a file, a form field, or a network packet, then the person on the other end of that channel chooses how far up the stack the write goes.

This is why "buffer overflow" is the standard name for the bug. The buffer is `buf`. It overflows when more bytes are written than it can hold, and the excess spills into whatever is next in memory.

Run It: AddressSanitizer Finds It

AddressSanitizer, usually shortened to ASAN, is a compiler feature that adds a bounds check to every memory access. It is turned on with one flag. Build the demo program with it and run it with a long argument:

```
cc -fsanitize=address -g 01-overflow.c -o 01-overflow-asan
./01-overflow-asan AAAAAAAAAAAAAAAAAAAAAAAAA
```

```
ERROR: AddressSanitizer: stack-buffer-overflow
WRITE of size 25 at 0x00016b4c2728
    #1 in vulnerable 01-overflow.c:20
    #2 in main 01-overflow.c:43

This frame has 1 object(s):
    [32, 40) 'buf' (line 16) <== Memory access at offset 40 overflows this variable
```

Reading the ASAN Report

Read the report from the top.

- **`stack-buffer-overflow`** is the category. ASAN also has categories for heap overflows, use of freed memory, and memory leaks.
- **`WRITE of size 25`** is the offending access. The input was 24 characters, and `strcpy` added the terminating zero, so it wrote 25 bytes.
- **`#1 in vulnerable 01-overflow.c:20`** is the line that did the writing. `#2` is its caller. This is a stack trace, the same list of frames you would see in `gdb` with `backtrace`.
- **`[32, 40) 'buf'`** says that `buf` occupies bytes 32 up to but not including 40 of the frame, so it is 8 bytes long. The write reached offset 40, which is the first byte past the end.

The `-g` flag is what makes the file and line numbers appear. Without it ASAN still catches the overflow but can only report addresses.

Without ASAN, the same program prints its output and then either crashes or does not, depending on how long the input was. The bug is the same either way. ASAN does not create the problem. It reports a problem that was already there.

Three Outcomes, One Bug

The same overflow produces different visible results depending only on how many bytes were written and what happened to be stored next to the buffer. There are three cases.

1. **Nothing visible.** The extra bytes landed in padding, or in a local variable the function had already finished using. The program runs to completion and prints the expected output. This is the most common outcome and the most dangerous one, because nobody notices. The program ships with the bug in it.
2. **A crash.** The extra bytes reached the return address, `ret` loaded a meaningless value into `%rip`, and the process was killed. This is annoying, but it is also honest. A crash in testing means the bug gets found and fixed.

3. **Control.** The bytes that landed on the return address were chosen on purpose, and they form a valid address. `ret` jumps there. The attacker now decides what code runs next, with whatever privileges the program had.

Why Tests Miss It

Only the third case is an attack, and all three come from the same line of C. Which one you get is not decided by the source code. It is decided by the input.

Notice what this means for testing. A test suite that runs the program with short, well-formed inputs will see case 1 every time and report success. The bug is only visible with an input that is long enough, which is exactly the input a tester would not think to try and an attacker would try first.

What the Attacker Actually Needs

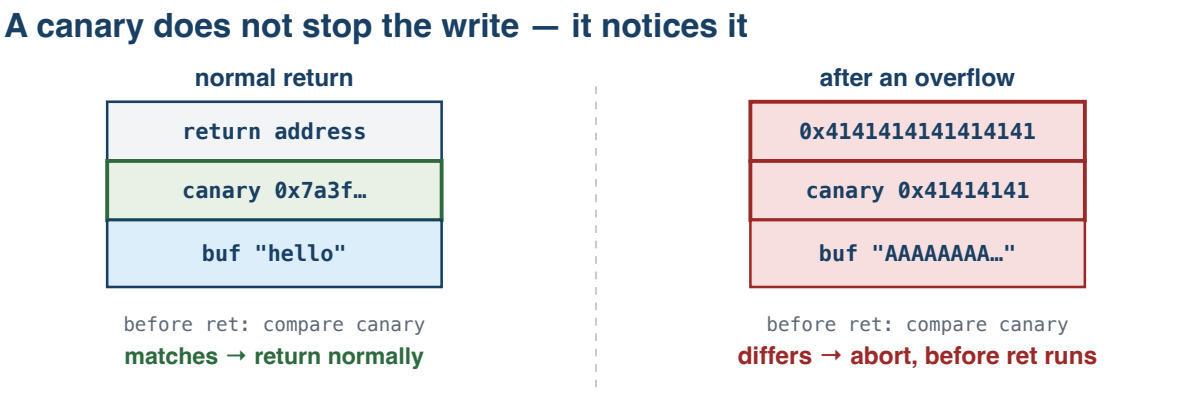
The list is short.

- **An input they control** that ends up in a fixed-size buffer. A command line argument, an environment variable, a file the program parses, a network message.
- **No bounds check** between that input and the buffer. A single `strcpy`, `gets`, `sprintf`, or a hand-written loop with the wrong limit.
- **Enough bytes** to reach the return address. The distance is fixed by the compiler's frame layout and can be measured by trial.

They do not need to find a mistake in your algorithm, guess a password, or understand what the program is for. The return address slot holds eight bytes, and `ret` will use whatever those eight bytes are. Lecture 7 stated this plainly: the return address is ordinary memory, and nothing checks it. This lecture is where that sentence pays off.

Once `%rip` is under the attacker's control, the classic next step was to point it at the buffer itself, which the attacker has also filled with machine code. The program then runs the attacker's instructions. The defenses in the next few slides were built, one at a time, to break the steps of that attack.

Defense 1: The Stack Canary



The canary is the first slide's secret value. The compiler places it in the frame between the local arrays and the return address. The idea is simple: an overflow that reaches the return address must pass through the canary on the way, and the write will change it.

How the Canary Check Works

In the prologue, the compiler stores a copy of a random value into the canary slot. The value is chosen once when the program starts, and it is different on every run. Just before `ret`, the compiler compares the slot with the original. If they still match, no overflow crossed that point, and the function returns normally. If they differ, the compiler's code calls `__stack_chk_fail`, which prints a message and aborts the process.

What the Canary Does Not Catch

Two properties matter.

- **It does not prevent the write.** By the time the check runs, the return address has already been overwritten. The canary turns a silent hijack into a loud crash, which is a large improvement, but the memory corruption still happened.
- **It only guards the return address.** A local variable that sits *below* the canary can still be overwritten by an overflow, and the canary will not notice. If that variable is a function pointer or a flag that controls a security decision, the attack still works.

The compiler flag `-fstack-protector-strong` turns canaries on for any function with a local array. `-fno-stack-protector` turns them off, which is how the classic textbook demonstration is built.

> **Warning**
>
> Whether canaries are on by default depends on how your compiler was configured when it was built, not on the C language. Check with `cc -O1 -S -o - prog.c | grep stack_chk`. If that prints nothing, the compiler emitted no canary code for that file.

Defense 2: ASLR

Address space layout randomisation, or ASLR, is an operating system feature. Each time a program starts, the OS picks a random base address for the stack, for the heap, and for each shared library and, if the program was built as position-independent code, for the program's own code as well.

The reason is the attack's last step. To take control, the attacker has to put a specific address into the return slot. In the classic attack that address pointed at the buffer. To reuse existing code, it points at a library function. Either way, the attacker has to know where the target is. If the target moves on every run, an address that was correct yesterday is wrong today, and the return jumps into unmapped memory or into the middle of an unrelated instruction.

ASLR: Same Binary, Two Layouts

```
         run 1          run 2
stack    0x16f1fa7b8    0x16d2e47b8
heap     0x101441510    0x13a80a510
code     0x100c045a8    0x104b285a8
```

Same binary, run twice, and all three regions moved. Look at the low digits: the addresses keep the same offset within a page, because the OS randomises where each region starts but not the layout inside it.

ASLR does not stop the overflow and does not detect it. It makes the attacker's guess of a target address unlikely to be right. A wrong guess crashes the program, and a crash is a much better outcome than a takeover.

Run It: Nothing Is Where It Was

The demo program `05-aslr.c` prints the address of a local variable, a `malloc` result, a function, and a global. Run it twice:

```
./05-aslr
./05-aslr
```

Compare the two outputs line by line. Every address that changed is in a region the OS randomised. On macOS and on a Linux system with the default settings, all four change.

This is also why the classic textbook exploit needs the flag `-no-pie` to demonstrate. `-no-pie` builds an executable that is loaded at a fixed address, which turns off randomisation for the program's own code. With randomisation on, the address the attacker writes into the return slot is only a guess, and a wrong guess is a crash rather than a takeover.

Defense 3: NX, and W^X

The oldest form of the attack put the payload in the buffer. The attacker typed machine code into the input, the overflow copied it onto the stack, and the overwritten return address pointed back at it. The stack was then executed as code.

The **NX bit** removes that option. NX stands for "no execute". Modern processors let the OS mark each page of memory as either executable or not, and the OS marks the stack as non-executable. If `%rip` is ever pointed at a stack address, the CPU refuses to fetch the instruction and raises a fault. The bytes the attacker wrote are still there, but they are data. They cannot run.

The general form of this rule is called **W^X**, read "write xor execute". A page of memory may be writable or executable, never both at once. Code pages are executable and read-only. Data pages, including the stack and the heap, are writable and never executable. There is no page where an attacker can both write bytes and later run them.

Like the other two defenses, this one changes what the attacker can do without touching the bug. The overflow still overwrites the return address. The attacker simply cannot use the buffer as the destination.

Defenses Are Layers, Not a Fix

Each defense blocks one step of the attack and leaves the others alone. Here is what each one stops and what it does not.

Defense	Stops	Does not stop
canary	overwriting the return address without being noticed	overwriting *other* locals below the canary
ASLR	a hard-coded target address	an attacker who first leaks an address and computes the layout from it
NX / W^X	executing bytes the attacker wrote	**reusing code that is already in the process**

Return-Oriented Programming

The last gap has a name: **return-oriented programming**, or ROP. If the stack cannot hold new code, the attacker chains together small fragments of code that already exist in the program and its libraries. Each fragment ends in `ret`, so if the stack holds a list of fragment addresses, each `ret` pops the next one and execution moves from fragment to fragment. The attacker never writes a single new instruction. Every instruction that runs was already in the binary.

So the picture is this. Each defense raised the cost of an attack, and together they made the simple version impossible. None of them reduced the cost to infinity, because the overflow itself is still there, and the overflow is the thing that hands the attacker the return address. The only way to remove the attack is to remove the overflow.

The Only Real Fix Is the Bound

The overflow happens because the copy stops when the input says to stop. The fix is to make it stop when the *destination* says to stop.

```c
char buf[64];

strcpy (buf, input);              // writes as many bytes as input contains
sprintf(buf, "%s", arg);          // same problem, different function

snprintf(buf, sizeof buf, "%s", arg);   // never writes more than sizeof buf
```

`snprintf` takes a size argument. It writes at most that many bytes, including the terminating zero, and it always writes the zero. If the input is longer than the space, the result is cut short, which is a wrong answer but not a memory corruption.

The size passed is `sizeof buf`. That expression is evaluated by the compiler at the point where `buf` is declared as an array, so the compiler knows its size. This is the one place in the program where the destination's size is available. Pass it along, and the copy is bounded by the destination. Forget to, and the copy is bounded by the input.

Two Ways the Bounded Copy Goes Wrong

Two ways this fix goes wrong in practice.

- **Passing the wrong size.** `sizeof` of a *pointer* to the buffer is 8, the size of the pointer, not the size of the array. Inside a function that received `buf` as a parameter, `sizeof buf` is 8. The size has to be passed as a separate argument from the place that declared the array.
- **Using a function that bounds the write but not the terminator.** The next slide covers the most common one.

The "Safe" Function That Is Not

`strncpy` looks like the bounded version of `strcpy`, and it is the fix that many programmers reach for first. It has the right shape: a destination, a source, and a size. But it was designed for a different job, filling fixed-width fields in old Unix directory entries, and its rules follow from that job.

`strncpy(dst, src, n)` copies at most `n` bytes. If `src` is shorter than `n`, it pads the rest of `dst` with zero bytes. If `src` is `n` bytes or longer, it copies exactly `n` bytes and stops. It does **not** add a terminating zero in that case, because a fixed-width field does not need one.

```c
char bad[8];
strncpy(bad, "0123456789", sizeof(bad));   // 8 chars, no NUL
```

After this call, `bad` holds `0`, `1`, `2`, `3`, `4`, `5`, `6`, `7` and nothing else. There is no zero byte in it. It is a `char` array, but it is not a C string, because a C string is defined by its terminating zero.

The next piece of code that treats `bad` as a string, such as `printf` with `%s` or `strlen`, starts reading at `bad[0]` and keeps going until it finds a zero byte. There is none inside `bad`, so it reads past the end, through whatever memory comes next. The bounded copy moved the overflow from the write to the read. It did not remove it.

Run It: `strncpy` Does Not Terminate

The demo program `02-strncpy.c` fills an 8-byte array with `#` characters first, so you can see exactly which bytes each function touched, then copies the 10-character string `"0123456789"` into it with each function.

```
strncpy into char[8]:
  bytes:   01234567
  no NUL in those 8 bytes -- printing it as a string is UB.

snprintf into char[8]:
  string:  0123456
  length:  7   (7 chars + NUL = 8)
```

`strncpy` wrote eight characters and stopped. All eight bytes are digits, so there is no terminator. The program

prints them one at a time with `putchar` in a loop, because printing the array with `%s` would read past its end.

`snprintf` Gives Up a Character to Terminate

`snprintf` wrote seven characters and then a zero byte. It gave up one character of the input to make room for the terminator, and the result is a valid string of length 7. The array can be printed with `%s`, passed to `strlen`, and copied again, all safely.

That is why the advice is `snprintf`, not `strncpy`. Both bound the write. Only one of them guarantees that the result is a string.

Undefined Behavior

The C standard is a document that says what each operation in the language means. For some operations, it says nothing. It declares them **undefined**, which means the standard places no requirements at all on what the program does if one of them is executed. The compiler may produce any code it likes for that case, including code that assumes the case never occurs.

Here are five common ones:

```
int x;            printf("%d", x);   // reading a variable that was never set
int *p = NULL;    *p = 42;           // writing through a null pointer
int i = 1;        int j = i++ + ++i; // changing i twice in one expression
int x = INT_MAX;  x = x + 1;         // signed integer overflow
arr[10]                              // reading past the end of int arr[10]
```

The last one is the buffer overflow from the first half of this lecture. It is not a special case. It is one entry on a list.

Signed Overflow Is Undefined, Unsigned Is Not

Signed overflow is on the list, and it is the one that surprises people most. Unsigned overflow is **not** undefined. The standard says that unsigned arithmetic wraps around modulo 2^N, so `UINT_MAX + 1` is exactly `0`, on every compiler, always. Signed arithmetic gets no such rule. `INT_MAX + 1` is undefined, and Lecture 5 kept insisting on that difference for the reason the next few slides show.

Defined, Implementation-Defined, Undefined

The standard actually distinguishes three levels of "the standard does not say exactly". Knowing which one applies tells you what you can rely on.

Level	What the standard promises	Example
implementation-defined	Each compiler picks one behavior and documents it. It is consistent within that compiler.	how many bits are in an `int`; whether `char` is signed
unspecified	The compiler picks from a set of allowed behaviors and does not have to document or be consistent.	the order in which function arguments are evaluated
undefined	No promise of any kind. The program has no meaning from that point on.	signed overflow, out-of-bounds access, null dereference

Why Undefined Is Different in Kind

The first two are portability concerns. A program that depends on `int` being 32 bits will break on a compiler where it is 16, but it does something sensible on each. You can find out what your compiler does and write code accordingly.

Undefined behavior is different in kind. There is no "what my compiler does" to find out, because the compiler is allowed to do something different on the next run, at the next optimisation level, or in the next version. And it is allowed to reason backward from the assumption that the undefined case does not happen. That reasoning is the subject of the next slide.

The Compiler Does Not Warn — It Assumes

This is the part that surprises people. You might expect that a compiler detects undefined behavior and reports it, the way it reports a syntax error. It does not. In most cases it cannot, because whether the behavior occurs depends on values that exist only at run time.

Instead the compiler takes the standard at its word. The standard says a correct program never executes undefined behavior. The compiler therefore assumes that the program in front of it is correct, and that every operation which would be undefined for some input never sees that input. Then it optimises on that assumption.

Follow the chain of reasoning for the most common case:

> Signed overflow is undefined, so a correct program never overflows `x + 1`. Therefore `x + 1` is always larger than `x`. Therefore the expression `x + 1 > x` is always true. Therefore an `if` on that expression always takes the true branch. The false branch is unreachable. Delete it.

Every step is valid given the previous one. The result is that a check you wrote to detect overflow is removed *because* it was written in terms of overflow. The compiler did not find a bug and hide it. It found an expression that is always true for every input a correct program could receive, and simplified it.

Two Mental Models, and Only One Is Compiled

The trap is that the programmer's mental model and the compiler's differ. The programmer thinks "on this machine, `INT_MAX + 1` wraps to `INT_MIN`, so the comparison will be false and catch it". The compiler thinks "the standard says nothing about what happens, so I need not preserve any particular result". Both are internally consistent. Only the compiler's model is what the generated code follows.

Run It: The Check That Disappeared

The demo program `03-ub.c` contains this function:

```
int check_overflow(int x) { return x + 1 > x; }
```

It is intended to return 0 when `x` is `INT_MAX`, because adding 1 would overflow. Build the same file twice, once at `-O0` and once at `-O1`, and run both:

```
=== -O0 ===                          === -O1 ===
check_overflow(INT_MAX)    = 0       check_overflow(INT_MAX)    = 1
check_overflow_ok(INT_MAX) = 0       check_overflow_ok(INT_MAX) = 0
check_unsigned(UINT_MAX)   = 0       check_unsigned(UINT_MAX)   = 0
```

Same source, same machine, same input. The first answer changed.

What Each Build Did

At `-O0` the compiler translates the source literally. It adds 1 to `x`, the addition wraps to `INT_MIN` on this hardware, the comparison runs and finds `INT_MIN > INT_MAX` false, and the function returns 0.

At `-O1` the compiler applied the reasoning from the previous slide. The whole function compiles to one instruction, `movl $1, %eax`, which returns 1 without looking at `x` at all. You can confirm this with `cc -O1 -S 03-ub.c` and reading the output.

The correct answer is 0. So your test passed in a debug build and failed in a release build, and no warning was printed at either level.

Why the Other Two Checks Survived

The other two lines did not change, and the reasons are instructive.

- `check_overflow_ok` is written as `return x != INT_MAX;`. It asks the same question, "is `x` the largest value?", without performing the addition. No undefined operation occurs, so the compiler has nothing to assume away.
- `check_unsigned` performs `x + 1 > x` on an `unsigned`. Unsigned overflow is defined to wrap, so the compiler must preserve the case where `x + 1` becomes 0, and the comparison stays.

The lesson generalises: to check for overflow, compare against the limit *before* the operation. Never perform the operation and then look at the result, because if the operation was undefined, the result is not required to mean anything.

The Linux Kernel, 2009

This is not a classroom curiosity. In 2009 a network driver in the Linux kernel contained code equivalent to this:

```
struct sock *sk = tun->sk;    // dereference tun
if (!tun)                     // ...then check tun for NULL
    return POLLERR;
```

Read it as the programmer did. The second line guards against a null `tun`. The programmer meant for the function to return an error if `tun` is null.

Now read it as the compiler did. The first line reads `tun->sk`, which dereferences `tun`. Dereferencing a null pointer is undefined. A correct program never does that, so in a correct program `tun` is not null at line one. Nothing changes `tun` between line one and line two. So at line two, `tun` is still not null, and the condition `!tun` is always false. The `if` is dead code. The compiler deleted it.

The Check That Was Not in the Binary

The compiled kernel therefore contained no null check at all. A user program that arranged for `tun` to be null could reach the dereference. And because the kernel, unlike a normal process, could be persuaded to map memory at address zero, the dereference did not crash. It read attacker-controlled data, and the bug became a local privilege escalation, which means an ordinary user could gain root.

The source code looked correct. The check was there, in plain text, and any reviewer reading it would have approved it. The compiled code did not contain it, because the dereference came first, and that order gave the compiler permission to remove the check. The fix was to move the check above the dereference, a change of one line.

Strict Aliasing

The compiler's job includes deciding when a value in memory might have changed, because if it has not changed, the compiler can keep using a copy it already loaded into a register instead of reading memory again. The C standard gives it a rule to decide this, called the **strict aliasing rule**: an object may only be accessed through a pointer of its own type, of a compatible type, or of a character type.

The consequence is that the compiler assumes a `float *` and a `uint32_t *` never point at the same object. If your code writes through one and reads through the other, the compiler is entitled to reorder those accesses, or to hand you the value it loaded earlier, because as far as the rule is concerned the two accesses cannot interfere.

```
float f = 1.0f;
uint32_t *p = (uint32_t *)&f;    // UB: reading a float through a uint32_t *
uint32_t bits = *p;
```

This compiles, and at `-O0` it does what you expect. At `-O2` it may not, and the failure is silent: `bits` holds a stale or partially updated value.

The Two Reads the Standard Allows

There are two ways to read the bits of a `float` that the standard defines.

```
uint32_t u;  memcpy(&u, &f, sizeof u);          // works in C and C++
union { float f; uint32_t u; } v; v.f = f;      // defined in C (not C++)
```

`memcpy` copies bytes. Character-type access is always allowed, and `memcpy` is specified in terms of bytes, so it is defined for any pair of objects of the same size. The compiler recognises this pattern and, at `-O1` and above, compiles it to a single register move. The call costs nothing.

A union with a `float` member and a `uint32_t` member is the other way. Writing `v.f` and then reading `v.u` reinterprets the same bytes. The C standard has said since C99 Technical Corrigendum 3 that this is defined. It is **not** defined in C++, and that difference is where the widespread belief that "unions are unsafe for type punning" comes from. In C, they are fine.

Run It: Two Defined Ways to Read the Bits

The demo program `04-aliasing.c` reads the bits of `3.0f` both ways:

```
f           = 3
bits_memcpy  = 0x40400000
bits_union   = 0x40400000
```

Both give `0x40400000`. That is the value you decoded by hand in Lecture 5b: sign bit 0, exponent field `10000000` for 2^1, and fraction `1.1` in binary for 1.5, giving $1.5 \times 2 = 3$. Lecture 5b's program used `memcpy` to get the bits without saying why. This is why.

Build with `-O1 -S` and look at the assembly for `bits_memcpy`. It is one `movd %xmm0, %eax` instruction: move the 32 bits from the floating-point register to the integer register. The `memcpy` call is gone. The compiler saw a 4-byte copy between two local objects and replaced it with the move that copies 4 bytes between registers. Writing the defined version cost nothing at run time.

The pointer cast is left commented out in the demo file on purpose. It would print the same value today, on this compiler, at this optimisation level. That is not the same as being correct, and the next compiler version is free to change it.

Tools That Find These

You will not find these bugs by reading the code. The kernel bug was reviewed by experienced people who did not see it. Use tools that check what the compiler actually does.

```
-Wall -Wextra -Werror            # warnings, treated as errors
-fsanitize=address               # overflows, use-after-free, leaks
-fsanitize=undefined             # signed overflow, bad shifts
-fsanitize=address,undefined -g  # both, with line numbers
```

What Each Tool Checks

- **Warnings** catch what can be seen at compile time: a variable used before it is set, a comparison between signed and unsigned, a format string that does not match its arguments. Turn them all on and treat them as errors.
- **AddressSanitizer** checks every memory access at run time against the bounds of the object it belongs to. It found the overflow on the first run and named the file, the line, and the variable.
- **UndefinedBehaviorSanitizer**, or UBSAN, inserts a check before each operation that could be undefined. When `x + 1` is about to overflow, it prints a message with the file and line instead of letting the compiler assume it away.

Both sanitizers make the program larger and roughly two to three times slower, so they are for development and testing, not for the build you ship. Run your tests under them anyway. A test suite that passes under ASAN

and UBSAN has checked every memory access and every arithmetic operation that the tests exercised, which is far more than a passing test suite normally means.

AI Systems Connection

A trained model is distributed as a file. That file comes from the internet, usually from someone you have never met, and it is loaded into a process with access to your GPU, your data, and your network. It is untrusted input, in exactly the sense this lecture has used the phrase.

For years, the usual way to load one was to *run* it. Python's `pickle` format, which PyTorch used for checkpoints, does not describe data. It describes a small **program** whose job is to reconstruct the data, and that program can call any Python function, including ones that open files or network connections. Loading a pickled checkpoint from a stranger is running their script. That is why the `safetensors` format exists: a JSON header that describes the tensors, followed by their raw bytes, with no mechanism for executing anything.

The Same Bug Under the Python

The parallel is exact. `strcpy` trusts that the input is short enough. `pickle.load` trusts that the file only wants to rebuild a tensor. Both hand control of your process to whoever wrote the input, and both were the default for a long time because they were convenient.

Underneath the Python, an inference server is built on C and C++: image decoders, video decoders, protobuf parsers, custom CUDA kernels. Each of those parses bytes from outside into fixed-size buffers. A malformed image that overruns a decoder's buffer is a buffer overflow in a network-facing process, which is the situation the first half of this lecture described.

The habit to take away: when input comes from outside, the question is never "does this work on the inputs I expect?" It is "what does this let the sender do?"

The Unit in One Slide

Three lectures, one machine. Each lecture added a skill and one fact that stayed true through all of them.

	what you learned	the thing that stayed true
L6	registers, `mov`, addressing modes, `lea`	nothing in the bits says what they are
L7	jumps, flags, the stack, `call` and `ret`	the return address is ordinary memory
L8	overflows, the defenses, undefined behavior	the machine does exactly what it is told

The Compiler Is Neither Adversary Nor Safety Net

Those three facts are the same fact seen from three sides. Memory holds bytes, not types. The bytes that decide where execution goes next are stored in the same memory as everything else. And the machine follows those bytes without asking where they came from.

The compiler sits between you and that machine. It is not your adversary, and it is not your safety net. It implements the language you actually wrote, as the standard defines it. The standard defines less than most programmers assume, and the gap between what you assumed and what it promises is where this lecture's bugs live.

Next unit: **the kernel**, and what happens when your program asks the operating system for something.

Practice

1. Build `01-overflow.c` with `-fsanitize=address` and run it with a long argument. What does ASAN report, and which line does it name? Now run it with a 9-character argument. What changes in the report?

2. Why can `strcpy` not check the size of its destination? What would have to change about C for it to be able to?
3. Run `03-ub.c` at `-O0` and `-O1`. Explain why the first answer changes and the other two do not.
4. Rewrite `if (x + 1 > x)` so it detects the same condition without relying on undefined behavior.

Practice II

5. Find the undefined behavior:

```
int table[10];
for (int i = 1; i <= 10; i++) table[i] = i;
```

6. Both `memcpy` and a pointer cast move the same bytes. Why is only one of them defined?

Practice III

7. A canary sits between the locals and the return address. Name a corruption it will not catch.
8. In the walk-through, the 25th byte was a zero. Where did it land, and why did that byte not cause the crash?

Practice — Answers 1–2

1. With the 24-character argument, ASAN reports a `stack-buffer-overflow`, a `WRITE of size 25` (24 characters plus the terminating zero byte), and names line 20 of `01-overflow.c`, the `strcpy` call in `vulnerable`. With a 9-character argument, only the size changes:

```
WRITE of size 10 at 0x00016f68a668
    #1 in vulnerable 01-overflow.c:20
[32, 40) 'buf' (line 16) <== Memory access at offset 40 overflows
```

The write still reaches offset 40, one byte past the end of `buf`, so ASAN still reports it. Even an 8-character argument is caught, because its terminating zero byte is the ninth byte written.

2. `strcpy` receives only a `char *`, and a pointer holds an address and nothing else. The size 8 is known to the compiler but is not stored anywhere at run time. For `strcpy` to check, the pointer would have to carry the size of the object it points to, or the function would need a size argument, as `snprintf` has.

Practice — Answers 3–4

3. At `-O0` the compiler translates `x + 1 > x` literally. The addition wraps to `INT_MIN` on this hardware, `INT_MIN > INT_MAX` is false, and the function returns 0. At `-O1` the compiler assumes signed overflow never happens, concludes that `x + 1 > x` is always true, and compiles the function to `movl $1, %eax`. The other two functions contain no undefined operation. `check_overflow_ok` compares `x != INT_MAX` and never adds. `check_unsigned` adds to an `unsigned`, and unsigned addition is defined to wrap, so the compiler must keep the comparison.

4. Compare against the limit before adding:

```
if (x != INT_MAX)       // x + 1 will not overflow
```

`x + 1 > x` is meant to be true exactly when `x` is not `INT_MAX`, so this asks the same question without doing the addition. The same idea works for any sum with a positive `y`. Check the limit first, then add:

```
if (x > INT_MAX - y)    // x + y would overflow
```

Practice — Answers 5–6

5. `int table[10]` has valid indices 0 through 9. The loop runs `i` from 1 through 10, so its last iteration writes `table[10]`, one element past the end. That write is undefined behavior, and it is the same bug as the buffer overflow in the first half of the lecture. The loop also never sets `table[0]`. That is not undefined, but it is probably not what was meant. The usual fix is to count from 0: `for (int i = 0; i < 10; i++)`.

6. The strict aliasing rule says an object may be read only through its own type, a compatible type, or a character type. The pointer cast reads a `float` object through a `uint32_t *`, which the rule forbids, so the compiler may assume the read cannot see the `float` and may reorder it or use a stale value. `memcpy` is specified as copying bytes, and byte access is always allowed, so it is defined for any two objects of the same size.

Practice — Answers 7–8

7. An overwrite of a local variable stored below the canary. An overflow of `buf` can change a neighboring local, such as a function pointer or a flag that controls a security check, without reaching the canary, so the check before `ret` still passes. A write that skips over the canary is also missed: `buf[i] = v` with a large `i` can change the return address directly and leave the canary untouched.

8. It landed at offset 24, the first byte above the return address, in memory that belongs to the caller's frame. It did not cause the crash because the crash happens first, at `ret`. The CPU tries to fetch an instruction from `0x4141414141414141`, and the process is killed. The caller never runs again, so nothing reads the byte that was changed. If the caller had resumed, that byte could have corrupted one of its values.

Reading

- CS:APP §3.10 (Buffer overflow, and the defenses against it)
- What Every C Programmer Should Know About Undefined Behavior